

APUNTESWEB · INGENIERÍA INFORMÁTICA



Segundo curso · 2.º cuatrimestre · 4 temas

JULIO DE 2026 · [HTTPS://ADRIANQL5.GITHUB.IO/APUNTESWEB/](https://adrianql5.github.io/apuntesweb/)

1. Comunicación y Sincronización entre Procesos

1.1 Comunicación entre Procesos (IPC)

En los sistemas con **multiprogramación** existe una necesidad de **comunicación entre procesos** (IPC) de forma bien estructurada sin usar **interrupciones**. Esta comunicación se usa para:

- **Transmitir información.**
- Evitar **interferencias** entre procesos.
- Garantizar el orden correcto cuando existen dependencias -> **sincronizarlos**

La comunicación entre **hilos** es prácticamente igual, pero la **transmisión de información** es mucho más **sencilla**, pues comparten espacio de direcciones.

1.1.1 Condiciones de Carrera

Los procesos que trabajan en conjunto pueden **compartir** cierto **espacio de almacenamiento** en el que pueden leer y escribir datos. Este almacenamiento compartido puede estar en la **memoria principal** (probablemente en el kernel) o en un **archivo**.

Una **condición de carrera** sucede cuando varios procesos acceden a datos compartidos y el resultado final depende de cuál y cuándo se ejecuta. **Depurar** programas que contienen condiciones de carrera es muy complicado, pues la mayoría de ejecuciones acaba con resultados incorrectos.

Ejemplo

Si se tiene en el kernel una **cola de impresión** en la que los procesos van escribiendo nombres de archivos cuando necesitan imprimirlos. Existen dos **variables compartidas** almacenadas en un archivo accesible por todos los procesos:

- **IN**, que apunta a la siguiente ranura libre de la cola.
- **OUT**, que apunta al siguiente archivo a escribir.

Hay un **demonio de impresión** que comprueba de forma periódica si hay archivos en la cola listos para imprimirse y, si los hay, imprime el primero.

- De manera más o menos simultánea, dos procesos (A y B) intentan poner en la cola un archivo.
- A lee **IN** y guarda el valor 7 en una variable local
- A pierde la CPU y se comienza a ejecutar B
- B lee **IN** y guarda el valor 7 en una variable local

- *B* almacena el nombre de un archivo que quiere imprimir en la posición 7 de la cola y actualiza **IN** a 8
- Se vuelve a ejecutar *A*
- *A* almacena el nombre de un archivo que quiere imprimir en la posición 7 de la cola y actualiza **IN** a 8
- Se ejecuta el demonio de paginación, no detecta nada extraño, pero el **archivo de *B* nunca se imprime**.

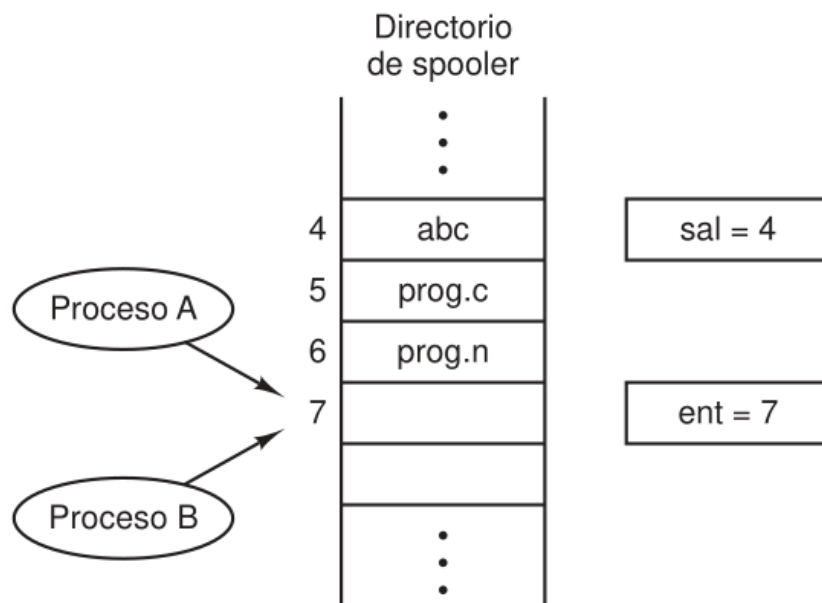


Figura 2-21. Dos procesos desean acceder a la memoria compartida al mismo tiempo.

1.2 Regiones Críticas

Para **evitar las carreras críticas** es necesaria una **exclusión mutua**, esto asegura que, si un proceso está utilizando datos compartidos, los demás no podrán acceder a ellos. Una **región crítica** es una parte del programa que accede a memoria compartida.

Para que varios procesos **cooperen en paralelo** de manera **correcta** y **eficiente** al usar datos compartidos se tienen que cumplir estas **condiciones**:

- No puede haber dos procesos en la **región crítica a la vez**: evita las **carreras críticas**
- No puede hacerse **suposiciones** acerca de las **velocidades** o el número de **CPUs**
- Un proceso no puede **bloquear** a otro **sin** estar en su **región crítica**.
- Ningún proceso debe **esperar indefinidamente** al acceso a la región crítica.

1.3 Exclusión Mutua con Espera Ocupada

Todas estas soluciones implican una **espera ocupada**, la espera ocupada es la **evaluación continua de una variable hasta que aparezca cierto valor**, lo cual desperdicia tiempo de CPU.

1.3.1 Deshabilitación de Interrupciones (hardware)

Consiste en que cada proceso **deshabilite todas las interrupciones** nada más entre a su región crítica y las rehabilite justo después de salir. Así, **no se realizarán cambios de contexto**, por lo que el proceso podrá acceder a la memoria compartida sin que ningún otro proceso intervenga.

El kernel deshabilita las interrupciones con frecuencia, pero es **peligroso** darle este poder a los procesos de usuario. **No** funciona en sistemas **multicore**, puesto que la deshabilitación solo afecta a la CPU que la ejecutó, permitiendo que las demás continúen trabajando y puedan acceder a la memoria compartida.

1.3.2 Variable Candado

Consiste en usar una **variable de candado compartida**, de manera que esta valga **1** cuando algún proceso está en su región crítica y **0** en caso contrario. Antes de entrar en sus regiones críticas, los procesos **comprobarán el valor del candado**, si es **0**, entrarán y lo establecerán a **1**. Si es **1**, esperarán.

Viola la condiciones de que no puede haber 2 procesos en la misma región crítica a la vez puesto que la variable de candado es **compartida** y es vulnerable a **carreras críticas**.

Ejemplo

El proceso *A* lee un **0** en el candado, pero antes de que lo fije a **1** se realiza un cambio de contexto y el nuevo proceso fija el candado a **1**. Cuando el proceso *A* se vuelva a ejecutar, también fijará el candado a **1** y por tanto ambos procesos estarán en sus regiones críticas a la vez.

1.3.3 Alternancia Estricta

Consiste en usar una **variable de turno compartida**, de manera que esta lleve la cuenta de a qué proceso le toca entrar a su región crítica. Al principio, **turno** vale **0**. Cuando el proceso *A* (turno **0**) desee entrar a su región crítica, verá que el valor de **turno** es **0** y entrará. El proceso *B* (turno **1**) también descubre que es **0** y se quedará evaluando **turno** continuamente hasta que cambie a **1**. Cuando **0** salga de la región, establecerá **turno** a **1**.

Sin embargo esto **viola** la condición de que **un proceso no puede bloquear a otro sin estar en su región crítica**. Además los procesos trabajan al **ritmo del más lento**.

1.3.4 Solución de Peterson

La implementación presentada es válida para mantener la exclusión mutua de 2 procesos, Se puede realizar para más, pero es mucho más compleja.

- Consiste en usar una **variable de turno compartida** y un **vector de 2 posiciones**, de manera que cada una de sus entradas informa de si el proceso al cual corresponde desea entrar en la región crítica.
- Cuando el proceso `0` desea entrar a su región crítica, invocará a `entrar_region()` indicando su interés en el vector y estableciendo `turno` a `0`. Como el otro proceso no desea entrar, accederá de inmediato. Si ahora el otro proceso también llama a `entrar_region()`, se quedará evaluando el vector continuamente hasta que `interesado[0]` cambie.

Cumple **todas las condiciones**, pero implica una **espera ocupada**.

Si el compilador cambie en `entrar_region()` el **orden** de la escritura en el **vector** y la en la variable `turno`, ambos procesos podrían estar dentro a la vez. Si ambos procesos invocasen `entrar_region()` más o menos a la vez, podrían aparecer **carreras críticas** en la variable `turno`, pero sólo entrará en la región aquel cuyo PID quede almacenado.

1.3.5 Instrucción TSL

Consiste en usar una **instrucción atómica** `TSL reg, lock` (*Test and Set Lock*), que lee el contenido de la palabra `lock` de memoria, lo guarda en un registro `reg` y después almacena un valor distinto de `0` en la dirección de memoria de `lock`.

Se garantiza que las operaciones leer y almacenar la palabra en memoria son **indivisibles**, es decir, ningún procesador puede acceder a la palabra de memoria hasta que termine la instrucción `TSL`.

Para conseguir esto, la CPU que ejecuta `TSL` **bloquea el bus de memoria**, impidiendo que otras CPUs accedan a ella hasta que acabe. Bloquear el bus de memoria es muy **distinto a desactivar interrupciones**, pues sí afecta al resto de procesadores. El **SO** y el **hardware** deben ayudar.

Instrucción XCHG

Funciona igual que la solución con `TSL`, pero usando la **instrucción atómica** `XCHG reg, lock` (*exchange*), que intercambia el contenido de la palabra `lock` de memoria con el registro `reg`. Es la implementación que usan las CPUs de **Intel**.

1.4 Dormir y Despertar

La solución de Peterson y las de `TSL` y `XCHG` son correctas, pero requieren una espera ocupada. La espera ocupada no sólo **desperdicia tiempo de CPU**, si no que también puede provocar un **problema**

de inversión de prioridades:

- Consideremos los procesos, H con prioridad alta y L con prioridad baja. El planificador ejecutará H siempre que esté listo.
- En instante en el que L está en su región crítica, H cambia a estado listo. Entonces, H se comienza a ejecutar, pero como L estaba en su región crítica, H se quedará en una espera ocupada indefinidamente, pues L nunca se planeará y por tanto nunca saldrá de la región crítica. (para lograr esto tienes que tener una planificación digna de un profundo subnormal, en la cual tengan prioridades que no decrezcan)

Ahora hablaremos de una primitiva de comunicación entre proceso que se **bloquean** cuando no pueden acceder a sus regiones críticas en lugar de desperdiciar tiempo de CPU. Se basa en las siguientes syscalls:

- `sleep`: hacer que el proceso llamador se bloquee hasta que otro proceso lo despierte
- `wakeup(PID)`: despierta al proceso de pid `PID`

1.4.1 Problema del Productor-Consumidor

Este problema se puede **generalizar** para aun número arbitrario de productores y consumidores **sin cambiar el código**.

- Dos procesos comparten un **buffer** común que puede contener N elementos. El **número de elementos** que tiene actualmente se almacena en `cuenta`
- El proceso **productor** coloca información en el buffer y el proceso **consumidor** la saca.
- Antes de introducir elementos en el buffer, el productor leerá `cuenta` para comprobar si el buffer está lleno. Si `cuenta = N`, el productor se duerme. En caso contrario, saca un elemento y decrementa `cuenta`.
- Ambos procesos comprueban si el otro se debe despertar y de ser así, lo despiertan.

Este método produce **condiciones de carrera** porque el **acceso a** `cuenta` **no está restringido**:

- EL buffer está vacío y el consumidor acaba de leer `cuenta` para ver si es `0`
- El planificador le retira la CPU al consumidor y se la cede al productor.
- El productor inserta un elemento en el buffer, incrementa `cuenta` y observa que ahora es `1`, por lo que llama a `wakeup` para el consumidor.
- Como el consumidor no se había dormido, la señal para despertarlo se pierde.
- El planificador le retira la CPU al productor y se la cede al consumidor.
- El consumidor evalúa el valor de `cuenta` que leyó antes. Como es `0`, se duerme.
- Tarde o temprano el productor llenará el buffer y se dormirá también. Ambos quedarán dormidos para siempre.

Para evitar esto, se podría usar un **bit de espera de despertar**, de manera que cuando se envía una señal de despertar a un proceso que no está dormido, se fija ese bit, y la próxima vez que ese proceso se intente dormir, no lo hará y restablecerá el bit. Esto funcionaría para este ejemplo simple, pero si hay más de un consumidor y/o productor un único bit será insuficiente.

1.5 Semáforos

Un **semáforo** es una variable entera que se usa para **contar** el número de **señales de despertar** guardadas para uso futuro. Puede tomar valor:

- `0`: no se guardaron señales de despertar
- `>0`: hay pendiente una o más señales de despertar

Esta solución se basa en las siguientes generalizaciones de `sleep` y `wakeup`:

- `down(semáforo)`: comprueba el valor del `semáforo`
 - Si es `0`: el proceso se pone a dormir sin completar la operación `down` por el momento.
 - Si es `>0`: disminuye el valor del `semáforo` (es decir, consume una señal de despertar almacenada) y continúa.
- `up(semáforo)`: incrementa el valor del `semáforo`
 - Si algún proceso estaba inactivo en ese `semáforo`, sin poder completar una operación `down` anterior, el sistema selecciona uno de ellos al azar y permite que complete la operación `down`. Así, después de la operación `up` en un `semáforo` con procesos dormidos, este seguirá en `0` pero habrá un proceso menos dormido en él.

Ambas operaciones son **atómicas**: una vez que empieza una operación de semáforo, ningún otro proceso podrá acceder a él hasta que acabe.

- Para asegurar esto, se implementan como **syscalls** en las que el SO **deshabilita las interrupciones** brevemente. En este caso la deshabilitación de interrupciones es aceptable pues el manejo del semáforo requiere sólo unas **pocas instrucciones**.
- Si se usan **varias CPUs**, cada semáforo debe estar protegido por una **variable de candado** sobre la cual se usen las instrucciones `TSL` o `XCHG`. En este caso la espera activa que implican estas instrucciones es justificable pues la operación de semáforo tarda sólo unos cuantos **microsegundos**.

La forma natural de **ocultar las interrupciones** a los procesos es asociar un semáforo que **inicialmente sea `0`** a cada dispositivo de E/S, de manera que:

- Justo después de iniciar el dispositivo, el **proceso administrativo** realiza un `down` en su semáforo, bloqueándose inmediatamente.
- Cuando entra la interrupción, el **manejador de interrupciones** realiza un `up` en el semáforo asociado, haciendo que el proceso administrativo despierte.

Los semáforos tienen dos **usos**:

- Los **semáforos binarios** son aquellos que se **inicializan a 1** y se usan para asegurar que sólo un proceso pueda entrar a su región crítica en un momento dado, es decir, para garantizar la **exclusión mutua**. Para usarlos, cada proceso realizará una operación `down` justo antes de entrar a su región crítica y una `up` después de salir de ella.

- Los **semáforos de sincronización** se usan para garantizar que ciertas secuencias de eventos ocurran o no.

1.5.1 Problema del Productor-Consumidor

Se usan 3 semáforos:

- `llenas`: contabiliza el número de ranuras llenas (inicialmente 0). Es un semáforo de **sincronización**
- `vacías`: contabiliza el número de ranuras vacías (inicialmente N). Es un semáforo de **sincronización**.
- `mutex`: asegura que el consumidor y productor no tengan acceso al buffer al mismo tiempo. Es un semáforo **binario**.

1.6 Mutexes

Un **mutex** es una variable con dos estados: **abierto** (desbloqueado) o **cerrado** (bloqueado). Se puede representar en un único bit, pero en la práctica se suelen representar con un entero con valor:

- `0`: abierto
- `!=0`: cerrado

Son **versiones simplificadas de los semáforos**, pues no tienen la habilidad de contar, sólo sirve para administrar la **exclusión mutua**. Se implementan con **facilidad** y **eficiencia** en espacio de usuario siempre que esté disponible la instrucción `TSL` o `XCHG`, por lo que son especialmente útiles en **paquetes de hilos implementados en espacio de usuario**.

Cuando un hilo (o proceso) necesita acceder a una región crítica llama a `mutex_lock`, de manera que si el mutex está abierto (es decir, la región crítica está disponible), puede entrar a la región. En caso contrario, se bloquea hasta que el que está en la región salga y llame a `mutex_unlock`. Si hay varios hilos bloqueados por el mutex al llamar a `mutex_unlock`, se selecciona uno de ellos al azar y se permite que entre a la región.

El código de `mutex_lock` es similar al de `entrar_region` de la solución con `TSL`, pero tienen una diferencia crucial:

- Cuando `entrar_region` no puede entrar a la región crítica, continúa evaluando el mutex en una **espera ocupada**. Si los hilos están a **nivel de usuario**, **no** hay un **reloj** que detenga a los hilos después de cierto tiempo, por lo que un hilo que intente entrar a una región crítica ocupada iterará indefinidamente y nunca adquirirá el acceso, pues nunca se ejecutará el hilo que está en la región crítica.
- Cuando `mutex_lock` no puede entrar a la región crítica, cede la CPU a otro hilo, por lo que **no** **hay espera ocupada**.

Ni `mutex_lock` ni `mutex_unlock` requiere syscalls, por lo que los hilos en nivel de usuario se pueden sincronizar completamente en **espacio de usuario**, usando procedimientos de unas **pocas instrucciones**.

1.6.1 Mutexes en Pthreads

`pthread` proporciona varias funciones para manejar mutexes:

- `pthread_mutex_init`: crea un mutex
- `pthread_mutex_destroy`: destruye un mutex
- `pthread_mutex_lock`: cierra un mutex
- `pthread_mutex_trylock`: trata de cerrar un mutex, pero si falla no bloquea al llamador, otorgándole la flexibilidad de decidir qué hacer a continuación (por ejemplo, realizar una espera ocupada).
- `pthread_mutex_unlock`: abre un mutex, liberando exactamente un hilo si hay alguno en espera.

Pueden tener **atributos**.

Variables de condición en Pthreads

Sirven para bloquear hilos hasta que se cumpla una determinada **condición**. Las variables de condición están **asociadas a un mutex**, de manera que cuando un hilo cierra el mutex y accede a la región crítica, puede quedar en espera por una variable de condición cuya condición no se cumple, hasta que otro hilo lo señale y pueda continuar.

`pthread` proporciona varias funciones para manejar variables de condición:

- `pthread_cond_init`: crea una variable de condición
- `pthread_cond_destroy`: destruye una variable de condición
- `pthread_cond_wait(&condp, &mutex)`: expulsa al hilo del mutex y lo bloquea hasta que algún otro lo despierta. El **bloqueo** del hilo y la **liberación del mutex** se realiza de forma **atómica**.
- `pthread_cond_signal(&condc)`: envía una señal a otro hilo bloqueado en la variable de condición para despertarlo.
 - Las variables de condición **no tienen memoria**, se se envía una señal a una variable de condición en la que no hay ningún hilo esperando, se pierde.
 - El hilo despertado volverá a competir por el mutex, por lo que, una vez lo adquiera, debe **comprobar si la condición que lo despertó sigue cumpliéndose**, pues desde que se envió la señal hasta la adquisición del mutex podría dejar de ser cierta.
- `pthread_cond_broadcast`: envía una señal a varios hilos para despertarlos

1.7 Monitores (esto no lo pregunta nide)

Aunque los **semáforos** pudieran parecer una manera muy sencilla de realizar la IPC, el programador debe ser extremadamente **cuidadoso** al usarlos. Para facilitar la escritura de programas correctos, se usa una primitiva de sincronización de **mayor nivel**, los monitores. Por ejemplo, en la solución al problema del productor-consumidor con semáforos, si se invierte el orden de las operaciones `down` del productos, llegará un punto en el que tanto el productor como el consumidor quedarán bloqueados de manera indefinida.

Un **monitor** es una colección de **procedimientos, variables y estructuras de datos** que se agrupan en un tipo especial de **módulo o paquete**. Los procesos pueden llamar a los **procedimientos** del monitor cuando lo deseen, pero no pueden acceder directamente a las **estructuras de datos internas** del monitor desde procedimientos ajenos a este.

Consiguen implementar **exclusión mutua** gracias a que solo permite que haya **un único proceso activo** en ellos e cada instante.

Son una **construcción del lenguaje de programación**, por lo que el compilador puede manejar las llamadas a sus procedimientos de manera especial:

- Cuando un proceso llama a un procedimiento de un monitor, sus primeras instrucciones comprobarán si hay algún otro proceso activo en él.
 - Si hay otro proceso activo en el monitor: el proceso llamador se suspende hasta que el otro abandone el monitor
 - Si no hay otro proceso activo en el monitor: el proceso llamador puede entrar
- Es responsabilidad del **compilador** implementar la exclusión mutua en las entradas del monitor, normalmente se usa un **mutex** o **semáforo binario**.

La **persona que escribe el monitor** no tiene que saber nada de cómo el compilador gestiona la exclusión mútua, sólo tiene que saber que, si convierte las regiones críticas en procedimientos del monitor, está garantizada. Entonces, como es el compilador (y no el programador) quien está gestionando la exclusión mutua, **es menos probable que algo salga mal**.

Se sigue necesitando una forma para que los procesos se bloqueen cuando no puedan continuar, para lo cual los monitores usan **variables de condición**:

- Para evitar tener dos procesos activos en el monitor a la vez, se necesita una regla que indique lo que ocurrirá **después** de la operación `signal`.
 - El proceso despertado se ejecuta y el llamador se suspende
 - El proceso llamador sale del monitor inmediatamente
 - Entonces, la instrucción `signal` sólo puede aparecer como instrucción final en un procedimiento de monitor.
 - El proceso llamador sigue ejecutándose, y hasta que salga del monitor el proceso despertado no podrá continuar.

Aunque `wait` y `signal` se parecen a `sleep` y `wakeup`, la exclusión mutua automática en los procedimientos de monitor garantiza que la instrucción `wait` se pueda completar sin preocuparse de que otro proceso intente realizar un `signal` antes de que se complete.

Los monitores son un **concepto de lenguaje de programación**, y muchos lenguajes no los tienen. En realidad estos lenguajes **tampoco** tienen **semáforos** en sí, pero se agregan fácilmente mediante una **biblioteca**. Los monitores no funcionan en **sistemas distribuidos** con varias CPUs con su propia **memoria privada** conectadas por una **red de área local**.

1.8 Pasaje de Mensajes

El **pasaje de mensajes** es un método de comunicación de procesos que se basa en las siguientes syscalls:

- `send(destino, &mensaje)`: recibe un mensaje a un destino especificado
- `receive(origen, &mensaje)`: recibe un mensaje de un origen especificado (o de **cualquier** origen, si se desea)

Si no hay un mensaje disponible, el llamador se puede **bloquear** hasta que llegue uno o puede regresar de inmediato con un **código de error**.

1.8.1 Aspectos de Diseño

Se deben **nombrar los procesos** de forma que los especificados como `origen` y `destino` estén **libres de ambigüedad**. Hay que utilizar algún **mecanismo de autenticación** para que los sistemas sepan que se están comunicando con quien esperan y no con un impostor.

Si los procesos que se están comunicando se encuentran en **distintas máquinas** conectadas por una red, se pueden **perder mensajes en dicha red**. Para protegerse de los mensajes perdidos, el emisor y el receptor puede acordar que, tan pronto como se reciba un mensaje, el receptor enviará un mensaje especial de **acuse de recibo: ACK**.

- Si el emisor no recibe el ACK en un cierto tiempo: **retransmite** el mensaje.
- Si el mensaje, se recibe correctamente, pero se pierde el ACK, el emisor retransmitirá, por lo que el receptor recibirá un **duplicado**.
 - Para que el receptor pueda diferenciar un mensaje nuevo de una retransmisión, se colocan **números de secuencia** consecutivos en cada mensaje original. Al recibir un mensaje con el mismo número de secuencia que otro anterior, el receptor sabrá que es un duplicado que debe ignorar.

Si los procesos que están comunicando se encuentran en la **misma máquina**, pueden aparecer problemas de **rendimiento**. **Copiar mensajes** de un proceso a otro es mucho más **lento** que realizar

una operación con un semáforo o un mutex. Como alternativa, se puede limitar el **tamaño de los mensajes** al de los **registros** de la máquina para realizar el pasaje usando los registros.

1.8.2 Problema del Productor-Consumidor

Se supone que todos los mensajes tienen el mismo tamaño, que no hay memoria compartida y que el SO coloca los mensajes enviados pero no recibidos automáticamente en el buffer.

- El consumidor envía N mensajes vacíos al productor.
- Cada vez que el productor tiene un elemento para dar al consumidor, recibe un mensaje vacío y envía de regreso uno lleno.

El **número total de mensajes** en el sistema permanece **constante** en el tiempo y se pueden almacenar en una **cantidad de memoria conocida** de antemano.

- Si el productor es más rápido: todos los mensajes acabarán llenos y el productor se bloqueará esperando un mensaje vacío.
- Si el consumidor es más rápido: todos los mensajes acabarán vacíos y el consumidor se bloqueará esperando un mensaje lleno.

Una manera de direccionar los mensajes consiste en asignar a cada proceso una dirección única y **direccionar los mensajes a los procesos**. Otra alternativa es usar un **buzón**, una estructura de datos que se usa como lugar para colocar un determinado número de mensajes (normalmente especificado en su creación).

- El productor y el consumidor crean buzones con el tamaño suficiente como para contener N mensajes:
 - El **productor** envía mensajes con los datos actuales al buzón del consumidor
 - El **emisor** envía mensajes vacíos al buzón del consumidor.
- Cuando un proceso trate de enviar a un buzón que está **lleno**, se **suspende** hasta que se elimine un mensaje de ese buzón.
- El **buzón** de destino de ambos **contiene** los mensajes que se han **enviado** al proceso destino **pero que todavía no se han aceptado**.

Otra forma de usar los buzones es para eliminar el uso del buffer completamente con la estrategia de **encuentro**:

- Si la operación `send` termina antes que la `receive`: el emisor se bloquea hasta que ocurre la operación `receive`, momento en el cual el mensaje se puede **copiar directamente** del emisor al receptor, **sin buffer de por medio**.
- Si la operación `receive` termina antes que la `send`: el receptor se bloquea hasta que ocurre una operación `send`.

Es más **fácil de implementar** que un esquema de mensajes con buffer. Pero es **menos flexible** debido a que el emisor y receptor se ven obligados a ejecutarse a paso de bloqueo.

1.9 Barreras

Este método de sincronización está destinado a **grupos de procesos**, en vez de las situaciones con dos procesos de tipo productor-consumidor. Algunas aplicaciones se dividen en **fases y tienen la regla** de que **ningún proceso puede continuar** a la siguiente fase hasta que **todos** los procesos **estén listos** para hacerlo.

Para lograr esto, se coloca una **barrera** ejecutando la primitiva `barrier`, **al final de cada fase**. Cuando un proceso llega ante la barrera, se **bloquea** hasta que todos llegan a ella.

1.10 Problemas clásicos del IPC

1.10.1 Problema de los lectores y escritores

El **problema de los lectores y escritores** modela el **acceso a una bd**. En un sistema con muchos procesos que intenta leer y escribir (actualizar) en una bd, puede haber **varios procesos leyendo** la bd a la vez, pero si un proceso la está **actualizando**, **ningún otro puede acceder** a ella (ni siquiera los lectores).

Una posible solución es que el **primer lector** en obtener acceso a la bd realice una operación `down` en el semáforo `bd`. Los **siguientes lectores incrementan un contador** `cl`.

A medida que los lectores van **saliendo**, **decrementan** `cl` y el último realiza un `up` en la `bd`, para permitir que un escritor bloqueado (si lo hay) entre. Mientras haya un lector activo, se admitirán los siguientes. Por tanto, siempre que haya un suministro continuo de lectores, todos entrarán nada más lleguen, por lo que un **escritor** que intente entrar estará **suspendido hasta que no haya un lector presente**.

Para evitar estas esperas en los escritores, se podría hacer que cuando llegue un lector y haya un escritor en espera, el **lector se suspenda detrás del escritor**. Así, cada escritor esperará a que terminen los lectores que estaban activos cuando llegó, pero no esperará a los que llegaron después de él. Así se logra una **menor concurrencia** y, por tanto, un menor rendimiento.

1.10.2 Problema de los Filósofos Comelones

El **problema de los filósofos comelones** modela procesos que compiten por el **acceso exclusivo** a un **número limitado de recursos**. Hay N filósofos sentados alrededor de una mesa circular, cada uno delante de un plato, que alternan entre comer y pensar. Para comer, necesitan dos tenedores. Hay un tenedor entre cada par de platos. Cuando un filósofo trata de comer, intenta adquirir sus tenedores izquierdo y derecho, **uno a la vez**, en cualquier **orden**. Si tiene éxito al adquirir ambos, come por un momento, deja los tenedores y después sigue pensando.

La solución más obvia sería que `tomar_tenedor` provoque una **espera** hasta que el **tenedor** esté disponible y luego lo tome. Pero esto no funciona, porque si todos los filósofos toman sus tenedores izquierdos a la vez, ninguno podrá tomar el derecho y habrá un **interbloqueo**.

Para evitar el interbloqueo, después de tomar un tenedor se **comprueba si el otro está disponible**. Si no lo está, el filósofo regresa el tenedor que había tomado, **espera** cierto tiempo y repite todo el proceso. Pero tampoco funciona, si todos los filósofos comienzan el algoritmo a la vez y toman sus tenedores izquierdos, verían que los derechos están ocupados, así que dejarían los izquierdos, esperarían, volverían a tomar todos a la vez los derechos, y así consecutivamente, entrando en **inanición**.

Para evitar la inanición se puede hacer que la **espera** tenga una **duración aleatoria**. Así la probabilidad de bloqueo prolongado es muy baja. Aunque sea poco probable, **puede fallar**, por lo que no sirve para aplicaciones que requieren seguridad total.

Para evitar el interbloqueo y la inanición se protegen las instrucciones que van después de la llamada a `pensar` mediante un **semáforo binario**. Antes de empezar a adquirir tenedores, el filósofo debe realizar un `down` sobre el. Esto tiene un error de rendimiento, porque **no permite la concurrencia**.

Para conseguir el máximo paralelismo posible se usa un **array** `estado` para llevar registro de si cada filósofo está comiendo, pensando o hambriento. Un filósofo sólo se puede mover al estado de **comer si ningún vecino está comiendo**. Para ello se usa un array de N semáforos de manera que los filósofos hambrientos puedan bloquearse si los tenedores que necesitan están ocupados.

2. Interbloqueos

2.1 Recursos

Los **recursos** son objetos que pueden ser otorgados por el SO a los procesos. Pueden ser **dispositivos hardware** o **piezas de información**. La secuencia de acciones requerida para utilizar un recurso es:

- Solicitar el recurso
- Usar el recurso
- Liberar el recurso.

Si el recurso **no está disponible** cuando se solicita, el proceso espera. En algunos SOs, esto implica que se **bloquea automáticamente** y se despierta cuando el recurso esté disponible. En otros, la **solicitud falla con un código de error** y el proceso puede decidir si va a esperar un poco y volver a intentarlo.

Aunque en el segundo caso el proceso no está bloqueado, es como si lo estuviera, pues se encuentra en un ciclo solicitando el recurso, pasando a estado inactivo y después intentándolo de nuevo.

Supondremos que **cuando a un proceso se el niega el acceso a un recurso pasa a estado inactivo**.

La naturaleza exacta de la solicitud depende del sistema. En algunos casos existe la syscall `request` para que los procesos pidan los recursos explícitamente. En otros, los únicos recursos que conoce el SO son los archivos especiales que sólo un proceso puede tener abierto a la vez (con la syscall `open` habitual).

2.1.1 Adquisición de Recursos

Para ciertos tipos de recursos, es **responsabilidad de los procesos** de usuario administrar su uso. Una manera de permitir esto es asociar un **semáforo binario o mutex** con cada recurso. Los tres pasos antes listados se implementan como:

- Una operación `down` en el semáforo para **adquirir el recurso**.
- Usarlo
- Finalmente realizar una operación `up` para liberarlo.

Cuando los procesos necesitan varios recursos, simplemente se adquieren uno después del otro. Ahora consideremos dos procesos que solicitan dos recursos.

- Si los procesos piden los recursos en el mismo orden no aparecerán interbloqueos.
- Si los procesos piden los recursos en distinto orden, pueden aparecer interbloqueos:

- Uno de los procesos adquiere su primer recurso e inmediatamente después el otro proceso adquiere el otro recurso. Entonces, ambos procesos se bloquearán cuando intenten solicitar el recurso que les falta.

2.2 Introducción a los Interbloqueos

Un conjunto de procesos está en **interbloqueo** si cada proceso del conjunto está esperando un evento que sólo puede ser ocasionado por otro proceso del conjunto. Como todos los procesos están en espera, ninguno producirá alguno de los eventos que podrían despertar a cualquiera de los otros, así que todos seguirán esperando para siempre.

Para este modelo supondremos que los procesos tienen **sólo un hilo** y que **no hay interrupciones** posibles para despertar a un proceso bloqueado. Si no hay interrupciones se evita que un proceso que de cualquier forma estaría en interbloqueo sea despertado por una interrupción y ocasione eventos que liberen a otros procesos del conjunto.

Normalmente se producen **interbloqueos de recursos**, en el que el evento por el que esperan los procesos es la **liberación de un recurso** que tiene otro miembro del conjunto.

2.2.1 Condiciones para los Interbloqueos de Recursos

Deben aplicarse cuatro condiciones para que suceda un interbloqueo de recursos:

- Condición de **exclusión mutua**: cada recurso se asigna en un momento dado a **sólo un proceso**, o está disponible.
- Condición de **contención y espera**: los procesos que actualmente contienen recursos que se les otorgaron antes pueden **solicitar nuevos** recursos.
- Condición **no apropiativa**: los recursos otorgados previamente no se pueden quitar a un proceso **por la fuerza**, deben ser liberados explícitamente por el proceso que los contiene.
- Condición de **espera circular**: debe haber una **cadena circular** de dos o más procesos, cada uno de los cuales espera un recurso contenido por el siguiente miembro de la cadena. Cada condición se relaciona con una **política** que puede tener o no un SO.

2.2.2 Modelado de Interbloqueos

Estas cuatro condiciones se pueden modelar mediante grafos dirigidos, los **grafos de recursos**.

- Tipos de **nodos**:
 - **Procesos**->círculos
 - **Recursos**->cuadrados
- Significado de **arcos**:
 - Arco de **recurso a proceso**: el recurso fue solicitado previamente por, y asignado a, y actualmente es contenido por ese proceso

- Arco de **proceso a recurso**: el proceso está actualmente bloqueado en espera de ese recurso.

Un **ciclo** en el grafo indica que hay **interbloqueo** que involucra a los procesos y recursos en el ciclo. Permiten ver si una secuencia de peticiones y liberaciones conduce al interbloqueo realizándolas paso a paso, y después de cada paso **comprobando** si el grafo tiene algún ciclo.

Por ejemplo, si tenemos los procesos A , B y C y tres recursos R , S y T . El SO es libre de ejecutar cualquier proceso desbloqueado en cualquier momento.

- Si se ejecutan los 3 procesos **secuencialmente**, uno detrás de otro, no aparecen interbloqueos (ya que no hay competencia por los recursos), pero **tampoco** tiene **paralelismo**.
- Si tenemos la siguiente ejecución **concurrente**, después de haber realizado la petición 4, A se bloquea en espera de S . Después, B y C se bloquean también. Aparece un ciclo, y, por tanto, un **interbloqueo**.

2.2.3 Estrategias para abordar los Interbloqueos

- **Ignorar** el problema
- **Detección y recuperación**: dejar que ocurran los interbloqueos, detectarlos y tomar acción.
- **Evitarlos dinámicamente**: asignar cuidadosamente los recursos
- **Prevención**: evitar estructuralmente una de las cuatro condiciones

2.3 Ignorar el problema: Algoritmo del Avestruz

El método más sencillo es **ignorar el problema** de los interbloqueos. Los costes asociados a intentar solucionarlos reducirían considerablemente el rendimiento del sistema, por lo que si los interbloqueos no son muy frecuentes suele ser preferible a no tomar acción contra ellos.

2.4 Detección y Recuperación de un Interbloqueo

En esta técnica, el sistema no trata de evitar los interbloqueos, en su lugar intenta detectarlos cuando ocurren y luego actúa para recuperarse de ellos.

2.4.1 Detección de Interbloqueos con un Recurso de cada Tipo

Se usará el grafo de recursos para determinar si existe un interbloqueo, por lo que se necesita un algoritmo para la **detección de ciclos** en un grafo dirigido. Se parte de una lista de nodos L y una lista de arcos que se irán marcando conforme se vayan explorando. Para cada nodo del grafo:

1. Para cada nodo N en el gráfico, realizar los siguientes cinco pasos con N como el nodo inicial.

2. Inicializar L con la lista vacía y designar todos los arcos como desmarcados.
3. Agregar el nodo actual al final de L y comprobar si el nodo ahora aparece dos veces en L. Si lo hace, el gráfico contiene un ciclo (listado en L) y el algoritmo termina.
4. Del nodo dado, ver si hay arcos salientes desmarcados. De ser así, ir al paso 5; en caso contrario, ir al paso 6.
5. Elegir un arco saliente desmarcado al azar y marcarlo. Después seguirlo hasta el nuevo nodo actual e ir al paso 3.
6. Si este nodo es el inicial, el gráfico no contiene ciclos y el algoritmo termina. En caso contrario, ahora hemos llegado a un punto muerto. Eliminarlo y regresar al nodo anterior; es decir, el que estaba justo antes de éste, hacerlo el nodo actual e ir al paso 3.

En otras palabras: se toman los nodos de uno en uno, como la raíz de lo que se espera sea un árbol, y se realiza una búsqueda de un nivel de profundidad en él. Si se regresa a un nodo que ya se encontró, hay un ciclo. Si agota todos los arcos de cualquier nodo, regresa al anterior.

Si regresa a la raíz y no puede avanzar más, el subgrafo que se puede alcanzar desde el nodo actual no tiene ciclos. Si esto se cumple para todos los nodos, no hay ciclos en el grafo entero, por lo que el sistema no está en interbloqueo.

2.4.2 Detección de Interbloqueos con Varios Recursos de cada Tipo

- Partimos de que existen n procesos, $P_1 \dots P_n$ y hay m tipos de recursos.
- E es el **vector de recursos existentes**, donde E_i es el número de instancias del recurso i existentes
- A es el **vector de recursos disponibles**, donde A_i es el número de instancias del recurso i disponibles en un momento dado.
- C es la **matriz de asignaciones actuales**, donde C_{ij} es el número de instancias del recurso j que están contenidas por el proceso i
- R es la **matriz de peticiones**, donde R_{ij} indica el número de instancias del recurso j que desea el proceso i . Se debe cumplir que cada recurso o esté asignado o esté disponible $\sum_{i=1}^n C_{ij} + A_j = E_j$ Para la comparación de vectores se usa la relación $A \leq B$, que indica que $A_i \leq B_i$ para $1 \leq i \leq m$

Todos los procesos están desmarcados al principio y se van marcando conforme progresa el algoritmo, indicando que pueden completarse. Cuando el algoritmo termine, todos los procesos no marcados están en interbloqueo.

1. Buscar un proceso desmarcado, P_i , para el que la i -ésima fila de R sea menor o igual que A .
2. Si se encuentra dicho proceso, agregar la i -ésima fila de C a A , marcar el proceso y regresar al paso 1.
3. Si no existe dicho proceso, el algoritmo termina.

Se supone que todos los procesos ****mantienen**** todos los recursos adquiridos ****hasta que terminan****. Aunque el algoritmo ****no es determinístico**** (ya que puede ejecutar los procesos en cualquier orden

posible), el ****resultado siempre es el mismo****.

En otras palabras: se busca un proceso que se pueda ejecutar hasta completarse, es decir, que tenga demandas de recursos que se puedan satisfacer con los recursos disponibles actuales. El proceso seleccionado se ejecuta hasta que termina y devuelve sus recursos a la reserva. Después se marca como completado.

Ahora que sabemos cómo detectar interbloqueos, surge la duda de **cuándo** buscarlos:

- Comprobar cada vez que se realiza una **petición** de un recurso. Detectaría los bloqueos lo más **pronto** posible, pero sería muy **costoso**.
- Comprobar cada x minutos
- Comprobar cuando haya disminuido el uso de la CPU por debajo de un mínimo. Si hay suficientes procesos en interbloqueo, habrá pocos ejecutables y la CPU estará inactiva con frecuencia.

2.4.3 Recuperación de un Interbloqueo

Recuperación por medio de apropiación

En ocasiones puede ser posible **quitar temporalmente un recurso** a su propietario actual y otorgárselo a otro proceso. En muchos casos esto puede requerir de intervención manual, sobre todo en SOs de procesamiento por lotes que se ejecutan en mainframes.

Poder hacer esto sin que el proceso expropiado lo note depende de la **naturaleza del recurso**: muchas veces es difícil o imposible

Recuperación a través del retroceso

Si se sabe que pueden aparecer interbloqueos, puede hacerse que los procesos realicen **puntos de comprobación** periódicamente escribiendo lo necesario en un archivo para reiniciarse más tarde. El punto de comprobación contiene la **imagen de la memoria** y qué **recursos** están asignados al proceso en este instante.

Para que sean más efectivos, los nuevos puntos de comprobación **no deben sobrescribir los anteriores**, sino que deben escribirse en nuevos archivos para que se acumule una secuencia completa conforme se vaya ejecutando el proceso.

Así, al detectar un interbloqueo, es fácil ver qué recursos se necesitan. Para realizar la recuperación, un proceso que posee un recurso necesario se revierte a un punto de comprobación antes de que lo adquiriera. Pero se pierde el trabajo realizado antes de el punto de comprobación.

Recuperación a través de eliminación de procesos

La forma más cruda y simple de romper un interbloqueo es **eliminar a uno o más procesos**. Se elimina uno de los **procesos en el ciclo**, a ver si se rompe el interbloqueo. Si no se rompe, se van eliminando

más hasta que suceda. También se puede eliminar un **proceso que no esté en el ciclo** para liberar sus recursos.

El proceso a eliminar se elige con cuidado, ya que está conteniendo recursos que necesita cierto proceso en el ciclo. Siempre que sea posible, es mejor eliminar un proceso que se pueda volver a ejecutar desde el principio **sin efectos dañinos**.

2.5 Evitar los Interbloqueos

Hasta ahora se supuso que los procesos solicitaban todos los recursos que necesitaban a la vez, pero en la mayoría de sistemas se **solicitan de uno en uno**. Entonces, el sistema debe ser capaz de **decidir si es seguro otorgar** un recurso cuando este es solicitado para poder evitar interbloqueos.

2.5.1 Trayectorias de los recursos

Este es un modelo para lidiar con dos procesos, A y B , y dos recursos, una impresora y un trazador.

- El eje **horizontal** representa el número de instrucciones ejecutadas por el proceso A y el **vertical** por el B .
- Cada **punto** en el diagrama representa un estado conjunto de ambos procesos.
 - Con un solo procesador, todas las rutas deben ser **horizontales o verticales**, nunca diagonales
 - El movimiento es hacia **arriba** o hacia la **derecha** (pues los procesos no pueden ejecutarse atrás en el tiempo).
- Uso de recursos de $A \rightarrow I_1 - I_3$ impresora, $I_2 - I_4$ trazador.
- Uso de recursos de $B \rightarrow I_6 - I_8$ impresora, $I_5 - I_7$ trazador.

Las regiones sombreadas son en especial interesantes. La región con las líneas que se inclinan de **suroeste a noreste** representa cuando **ambos procesos tienen la impresora**. La regla de exclusión mutua hace imposible entrar a esta región.

De manera similar, la región sombreada de la otra forma representa cuando ambos procesos tienen el trazador, y es igual de imposible.

Si el sistema entra alguna vez al cuadro delimitado por $I1$ e $I2$ en los lados, y por $I5$ e $I6$ en la parte superior e inferior, entrará en interbloqueo en un momento dado, cuando llegue a la intersección de $I2$ e $I6$. En este punto, A está solicitando el trazador y B la impresora, y ambos recursos ya están asignados. Todo el cuadro es inseguro y no se debe entrar en él.

En el punto t , lo único seguro por hacer es ejecutar el proceso A hasta que llegue a $I4$. Más allá de eso, cualquier trayectoria hasta u bastará. Lo importante a considerar aquí es que en el punto t , B está

solicitando un recurso.

El sistema debe decidir si lo otorga o no. Si se otorga el recurso, el sistema entrará en una región insegura y en el interbloqueo, en un momento dado. Para evitar el interbloqueo, B se debe suspender hasta que A haya solicitado y liberado el trazador.

2.5.2 Estados Seguros e Inseguros

En cualquier instante hay un **estado actual** que consiste en E , A , C y R (esto se explicó antes).

Se dice que un estado es **seguro** si hay cierto **orden de programación** en el que se puede **ejecutar cada proceso hasta completarse**, incluso si todos ellos solicitasen repentinamente su número máximo de recursos de inmediato. Ejemplo: se tienen los procesos A , B y C y 10 instancias de un mismo recurso:

1. Estado de la figura A es **seguro**, ya que existe una secuencia de asignaciones que permite completar todos los procesos
 2. Si se ejecuta B exclusivamente, hasta que pida y obtenga dos instancias más del recurso, se llega a la figura B.
 3. Al completarse B , se llega a la figura C.
 4. Si después de ejecuta C de la misma manera, se llega a la figura D.
 5. Cuando C se complete, se obtiene la figura E
 6. Ahora A puede obtener las seis instancias del recurso que necesita y también completarse.
-
1. Si partimos de la figura A anterior, A solicita y obtiene 1 recurso, se llega a la figura B, que muestra un estado **inseguro**.
 2. Si se ejecuta B exclusivamente, hasta que pida y obtenga todos sus recursos, se llega a la figura C.
 3. Al completarse B , se llega a la figura D.
 4. Ahora no hay manera de continuar, sólo hay 4 instancias libres y ambos procesos activos necesitan 5 de cada uno. Tampoco funcionaría si a partir del paso 0 se ejecutase A o B . Entonces, el paso de la figura A a la B supuso convertir un estado seguro a uno inseguro.

Un **estado inseguro no es un estado de interbloqueo**, pues el sistema puede ejecutarse durante cierto tiempo. La diferencia entre un estado seguro y uno inseguro es que, desde un estado seguro, el sistema puede **garantizar** que todos los procesos terminarán. Desde uno inseguro no se puede dar esa garantía.

2.5.3 Algoritmo del Banquero para un solo Tipo de Recurso

El algoritmo **considera cada petición** que va sucediendo y analiza si al otorgarla se produce un estado **seguro**. Si es así, se otorga. En caso contrario, se **pospone** hasta más tarde. Para ver si un estado es seguro, se comprueba si tiene suficientes recursos para satisfacer a un proceso. De ser así, se asume

que se devuelven los recursos otorgados a ese proceso y se comprueba el siguiente más cercano al límite. Si todos podrán satisfacerse en algún momento, el estado es seguro.

Ejemplo: cuatro procesos A , B , C y D y 10 instancias de un mismo recurso disponible.

1. Todos los procesos saben de antemano cuántos recursos van a necesitar (figura A).
2. Si se llega a la situación de la figura B, se habrá alcanzado un estado **seguro** (se puede ejecutar C hasta que termine y libere sus recursos, después D o B , y así sucesivamente).
3. Si a partir de la figura B se otorga una petición de B para una unidad, se llegaría a la figura C, que muestra un estado **inseguro** (si todos los procesos pidieran repentinamente todos los recursos que necesitan no se podría satisfacer a ninguno).

Un estado **inseguro no tiene por qué conducir a un interbloqueo**. Puede ser que a partir de la figura C algún proceso solicite una cantidad de recursos menor a su máximo.

El algoritmo del banquero considera cada petición a medida que va ocurriendo, y analiza si al otorgarla se produce un estado seguro. Si es así, se otorga la petición; en caso contrario, se pospone hasta más tarde. Para ver si un estado es seguro, el banquero comprueba si tiene los suficientes recursos para satisfacer a algún cliente.

2.5.4 Algoritmo del Banquero para Varios Tipos de Recursos

Además de E , A , C y R , se muestra P , el **vector de recursos poseídos**, que es la diferencia de E y A . Se supone que todos los procesos **mantienen** todos los recursos adquiridos **hasta que terminan**. Se supone que los procesos declaran sus **necesidades totales** de recursos antes de empezar a ejecutarse.

El algoritmo para determinar si un estado es seguro es el mismo que se usaba para detectar interbloqueos con varios tipos de recursos:

1. Buscar un proceso desmarcado, P_i para el que la fila i de R sea $\leq A$.
2.
 - Si se encuentra dicho proceso: Se agrega la fila i de C a A y se marca el proceso para regresar al paso 1
 - Si no existe, termina
3. El algoritmo finaliza
 - Cuando todos los procesos estén marcados: estado inicial era seguro
 - No se encuentra ningún proceso desmarcado que cumpla la condición del paso 1: hay interbloqueo.

Ejemplo: 1. EL estado de la figura es seguro 2. Si el proceso \$B\$ pide una impresora, se le puede otorgar ya que el estado resultante también es seguro (\$D\$ puede terminar, y después \$A\$ o \$E\$, seguidos por el resto) 3. Si después \$E\$ solicita la otra impresora, llevaría a un interbloqueo, así que la petición de \$E\$ se retrasa unos momentos.

Los procesos raras veces saben de **antemano** cuáles son sus **máximas necesidades** de recursos. El **número de procesos no es fijo**, varía dinámicamente a medida que los nuevos usuarios inician y cierran sesión. Los recursos que se consideraban disponibles pueden **desvanecerse**. Por ello pocos sistemas existentes usan el algoritmo del banquero para evitar interbloqueos.

2.6 Prevenir los Interbloqueos

Entonces, **evitar los interbloqueos es imposible**, pues **se requiere información sobre peticiones futuras que no se conocen**. En su lugar, los interbloqueos se pueden prevenir asegurando que por lo menos una de las **condiciones** para que sucedan nunca se cumpla.

NOTA

Evitar es q no haces nada para q haya menos interbloqueos pero intentas solucionarlo antes d q ocurran y prevenir es atacar las características d los interbloqueos para q no ocurran directamente

2.6.1 Cómo atacar la condición de Exclusión Mutua

Si ningún recurso se puede asignar exclusivamente a un único proceso, nunca habrá interbloqueos. Naturalmente, para que varios procesos puedan contener una impresora a la vez, no se les puede permitir escribir en ella al mismo tiempo. En su lugar, se coloca la salida de la impresora en una **cola de impresión**, de manera que varios procesos puedan generar la salida al mismo tiempo. Así, el único proceso que realmente solicita la impresora física es el **demonio de impresión** que nunca solicitará otros recursos, por lo que ya no aparecerán interbloqueos para la impresora.

Si el demonio se programa para comenzar a **imprimir** aunque **no todo el archivo** esté en la cola, la impresora podría permanecer inactiva largos períodos de tiempo si un proceso decide esperar después de la primera ráfaga de salida en el archivo que envió a la cola. Para evitar esto, los demonios normalmente se programan para imprimir sólo después de que esté disponible el **archivo de salida completo**. Esto podría provocar un **interbloqueo**: si dos procesos llenan cada uno la mitad de la cola de impresión pero ninguno termina de producir su salida completa, ninguno de los 2 podrá continuar.

Aún así, se puede extraer una idea útil de este concepto: evitar asignar un recurso cuando no sea absolutamente necesario y tratar de asegurarse de que la menor cantidad posible de procesos soliciten este recurso.

2.6.2 Cómo atacar la condición de Contención y Espera

Si se impide que los procesos que contienen recursos esperen por más recursos, nunca habría interbloqueos. Una manera de lograr esto es requerir que todos los procesos **soliciten todos sus recursos antes de empezar** a ejecutarse. Si todo está disponible, se le asignará y podrá ejecutarse hasta el final. Si no está todo disponible, no se asignará nada y el proceso esperará.

- Tiene el mismo problema que el **algoritmo del banquero**: la mayoría de procesos no saben cuántos recursos necesitarán antes de ejecutarse.
- Los recursos no se usan de manera **óptima**.

Aún así, algunos sistemas de procesamiento por lotes de mainframes lo usan, requiriendo al usuario que liste todos los recursos necesarios en la primera línea de trabajo.

Otra manera de conseguir esto es requerir que un proceso que **solicita** un recurso **libere** temporalmente todos los recursos que contiene en ese momento. Después puede tratar de obtener todo lo que necesite de una vez.

2.6.3 Cómo atacar la Condición no Apropiativa

Si permite retirarle temporalmente un recurso por la fuerza a un proceso sin dañar su ejecución, nunca habría interbloqueos. Muchos recursos son **no apropiativos**, así que retirárselos a los procesos sin dañarlos no es posible. Ciertos de estos recursos se pueden **virtualizar** para evitar esta situación:

- Al usar una **cola de impresión** en el disco con un **demonio de impresión** para manejar la impresora real, se eliminan los bloqueos que la involucran.
- Sin embargo, aparece la posibilidad de **interbloqueo** por espacio en el **disco**. Con discos grandes es muy **difícil** llegar a un interbloqueo por esto, ya que es difícil que se queden sin espacio disponible. No todos los recursos se pueden virtualizar. Por ejemplo: los registros de las bd o las tablas del SO se deben bloquear para usarse, de lo cual puede surgir un interbloqueo.

2.6.4 Cómo atacar la condición de Espera Circular

Si se impide que aparezca una cadena circular de varios procesos en espera de recursos que contienen otros procesos de la cadena, nunca habrá interbloqueos.

Un método es establecer una regla que diga que un proceso tiene derecho sólo a **un recurso en cada momento**. Si necesita otro, debe liberar el primero. Naturalmente, esto haría imposible ejecutar ciertos procesos.

Otro método es proporcionar una **numeración global** de todos los recursos, de manera que los procesos pueden solicitar recursos cada vez que quieran, pero todas las peticiones se deben realizar en orden numérico.

Así, el grafo de asignación de recursos **nunca** puede tener **ciclos**. Desmotivación:

- 2 procesos y 2 recursos: partiendo de que A contiene el recurso i y el B el j , se obtendrá un interbloqueo sólo si A solicita el recurso j y B el i .
 - Si $i > j \rightarrow A$ no podrá solicitar j , pues es menor que el que ya contiene.
 - Si $j > i \rightarrow B$ no podrá solicitar i , pues es menor que el que ya contiene.
 - Entonces, el interbloqueo es imposible.

- Con un número arbitrario de procesos y recursos: en cada instante, uno de los recursos asignados será el más alto. El proceso que lo contenga nunca pedirá un recurso que ya está asignado. En un momento dado, terminará y liberará sus recursos. En ese punto, otro proceso contendrá el recurso más alto y también podrá terminar.
 - Entonces existe un escenario en el que todos los procesos terminan, así que no hay interbloqueo.

Una variación del método anterior es retirar el requerimiento de que los recursos se adquieran en una secuencia cada vez más estricta, y simplemente obligar a que ningún proceso pueda solicitar un **recurso con numeración menor que los que ya contiene**.

El ordenamiento numérico soluciona el problema, pero puede ser **imposible encontrar un ordenamiento** que satisfaga a todos los procesos cuando el número de recursos potenciales y usos distintos es muy grande.

3. Sistemas en Tiempo Real

Un **sistema en tiempo real** recibe estímulos desde dispositivos físicos externos y su buen comportamiento implica:

- Responder correctamente a estos estímulos
- Las respuestas se producen dentro de un intervalo de tiempo determinado

Ejemplos: reproductor de CDs de audio, monitores de pacientes en las urgencias de un hospital, autopiloto en una nave, etc.

Hay dos tipos de **sistemas en tiempo real**:

- Sistemas de **tiempo real duro**: el tiempo de respuesta debe garantizarse a toda costa
- Sistemas de **tiempo real suave**: una respuesta tardía no produce graves daños, pero sí un deterioro del funcionamiento global.

3.1 Planificación en Sistemas de Tiempo Real

El comportamiento en tiempo real se logra dividiendo el programa en varios procesos que tienen:

- Comportamiento predecible y conocido de antemano
- Tiempos de vida cortos, y su ejecución finaliza en muchos menos de 1 segundo.

Cuando se detecta un nuevo evento externo, es responsabilidad del planificador planificar los procesos de tal forma que se cumpla con todos los tiempos límite. Los eventos a los que puede llegar a responder un **sistema en tiempo real** son de dos tipos:

- **Periódicos**: ocurren a intervalos regulares
- **Aperiódicos**: ocurren de manera impredecible

Si un sistema tiene que atender a n eventos periódicos, el **sistema en tiempo real** será **planificable** si:

- $\sum_{i=1}^n \frac{C_i}{T_i} = U \leq 1$ donde
- $n \rightarrow$ número de eventos periódicos.
- $U \rightarrow$ factor de utilización.
- $C_i \rightarrow$ tiempo máximo de ejecución para el evento i .
- $T_i \rightarrow$ periodo para el evento i .
- $\frac{C_i}{T_i} \rightarrow$ fracción de CPU que está utilizando el evento i .

En este cálculo se supone que la sobrecarga por cambio de contexto es tan pequeña que se puede ignorar.

Los algoritmos de planificación en tiempo real puede ser:

- **Estáticos:** toman sus decisiones de planificación antes de que el sistema empiece a ejecutarse. Solo funciona cuando hay disponible de antemano información perfecta acerca del trabajo que se va a realizar y los tiempos límite que se tienen que cumplir
- **Dinámicos:** toman sus decisiones de planificación en tiempo de ejecución.

3.2 RMS: Programación Monotónica en Frecuencia

Es un algoritmo de programación de tiempo real **estático**. Se puede usar para procesos que:

- Cada proceso periódico se debe completar dentro de su periodo
- Ningún proceso es dependiente de otro
- Cada proceso necesita la misma cantidad de tiempo de CPU en cada ráfaga
- Ningún proceso no periódico tiene tiempo de respuesta
- Los cambios de contexto ocurren instantáneamente, sin sobrecarga.

Se le asigna a cada proceso una prioridad fija, igual a la frecuencia de ocurrencia de su evento de activación. Por ejemplo: un proceso que se debe ejecutar cada 30 meseg tiene prioridad 33.

El planificador ejecuta **siempre** el proceso que esté listo y tenga la mayor prioridad, reemplazando el proceso en ejecución si es necesario. Es el algoritmo estático óptimo. Por ejemplo: los procesos A , B y C tiene prioridades estáticas de 33, 25 y 20, respectivamente. Por tanto, C tendrá que esperar a que la CPU esté inactiva para poder ejecutarse.

3.3 EDF: Menor Tiempo de Respuesta Primero

Es un algoritmo de programación de tiempo real **dinámico**.

- Igual que RMS, se puede usar para procesos aperiódicos
- A diferencia de RMS, no requiere el mismo tiempo de ejecución en cada ráfaga de CPU
- Cada proceso periódico se debe completar dentro de su periodo

Cada vez que un proceso necesita tiempo de CPU, anuncia su presencia y su tiempo de respuesta. El planificador mantiene una lista de procesos ejecutables ordenados según su tiempo de respuesta. Ejecuta el primero en la lista (el que tiene el tiempo de respuesta más cercano). Cada vez que un nuevo proceso está listo, se comprueba si su tiempo de respuesta ocurre antes que el del proceso en ejecución. De ser así, el nuevo proceso reemplaza al actual.

¿Por qué falla RMS? Porque ese algoritmo solo funciona cuando se cumple que:

$$\sum_{i=1}^n \frac{C_i}{T_i} \leq n(2^{1/n} - 1) \equiv n = 3, \text{ } usoCPU \leq 0.780$$

Dado que en el ejemplo anterior el uso de CPU era de 0.975, *RMS tenía muchas probabilidades de fallar*, mientras que por el contrario, **EDF siempre funciona para cualquier conjunto programable de procesos.**

5. Shell y Scripts

5.1 Introducción

El **shell** o intérprete de comandos es la **interfaz** del usuario con el SO, se encarga de traducir **líneas de comandos** en **solicitudes de acción del SO**. La **programación shell-script** es una **secuencia de comandos** de shell que sirve para la administración del sistema, programación de tareas repetitivas,...

En **unix** el shell **no forma parte del kernel**. Hay varios shells para unix:

- sh, csh, tcsh, ksh, powershell, etc.
- bash: shell por defecto en Linux. Tiene todas las características de sh e incluye características avanzadas. La sintaxis es muy parecida entre shells. sh tiene compatibilidad hacia delante

5.1.1 Funcionamiento del Shell

El shell es un proceso **iterativo** que ejecuta comandos de 2 tipos:

- Comandos **internos** al shell: se ejecutan en el **mismo proceso** y su código son **funciones** del código del shell. `cd`, `bg`, `alias`, `pwd`, `echo`, `eval`, ...
- Comandos **externos** al shell: inician un **nuevo proceso** y cambian su imagen por la del archivo ejecutable, su código es **independiente** del shell (está en los directorios indicados en la variable de entorno `$PATH`). `cp`, `cat`, `grep`, `mkdir`,...

`type comando` : averigua el tipo de un comando

5.2 Línea de comandos

Los comandos suelen tener 3 componentes: `comando [opciones] [parámetros]`. Se pueden ejecutar de 2 maneras:

- Ejecución **en primer plano**: el shell espera (waitpid) a que termine el comando antes de aceptar uno nuevo.
 - `ctrl + z` : finaliza el comando
 - `ctrl + c` : pausa el comando
- Ejecución en **segundo plano**: (`comando &`) el shell no espera a que termine el comando antes de aceptar uno nuevo.
 - `jobs` : permite ver la lista de comandos en segundo plano
 - `fg PID` , `bg PID` : permiten mover un comando a primer plano o a segundo plano

Los comandos tienen un código de salida que se almacena en `$?`:

- Si es igual a `0`, el comando terminó bien
- Si no es igual a `0`, el comando terminó con error
- `comando1 && comando2`: el segundo comando solo se ejecuta si el `$?` del primero es `0`
- `comando1 || comando2`: el segundo comando solo se ejecuta si el `$?` del primero es distinto de `0`
- `history`: devuelve una lista de comandos que se pueden volver a ejecutar con :
 - flechas
 - !numero: si lo ejecutas te escribe el comando que se corresponde con ese numero en el historia
 - `ctrl + r` permite hacer una búsqueda inversa
 - `fc i j` permite repetir varios
- `man [comando]`: manual
- `comando help`: manual de comandos internos (solo funciona en bash)
- `info comando`: manual más flexible o completo en hipertexto
- `echo`: para mostrar una línea de texto
- `cat/head/tail/grep/find/sort fichero` sirven para el procesamiento de texto

5.2.1 Expansión de Ficheros

Los **comodines** son caracteres especiales que sustituyen a otro/s para especificar varios ficheros como parámetro.

- `*` cualquier cadena (incluso vacía)
- `?` cualquier caracter
- `[x,x,x,...]` uno de los indicados
- `[!x,x,x,...]` o `[^x,x,x,...]` cualquier caracter excepto los indicados

5.2.2 Otras Expansiones

- Expansión de **llaves** (`{x,x,x}...`) para generación de strings (`echo a{1,2,3}b --> a1b a2b a3b`)
- Expansión de **tilde** para el directorio raíz de usuario (`$HOME`)
- Expansión **aritmética** (`$((expresion))` o `$(expresion)`) para evaluar expresiones (`echo $[(4+11)/3]`)

5.2.3 Variables de Shell

Se puede **crear** variables desde la línea de comandos con:

- `nombre='algo'` (sin espacios entre el =)
- `read nombre [Enter]` (read para el primer \n que encuentre, sólo lee la primera línea)

Se pueden **combinar** texto y variables: `'... $nombre ... '` (no usar "")

- **Variables locales:** visibles sólo desde el shell actual
- **Variables de entorno:** `export NOMBRE` para que sean visibles en todos los shells hijo
- `...$nombre...` para acceder al contenido de una variable
- `nombre=` vaciar una variable
- `printenv` lista las variables

5.2.4 Caracteres Especiales

Son caracteres que el shell trata de forma especial:

- `&, *, ?, [], [!], $, ;, <, >, <<, >>, \`
- `'...'` ignora todos los caracteres especiales
- `"..."` ignora todos los caracteres especiales excepto `$, \, '` (y la tilde torcida)
- `\` ignora el carácter especial que le sigue.

5.2.5 Redirección de Entrada y Salida

Toda la E/S se realiza a través de ficheros. Cada proceso tiene asociado 3 **ficheros de E/S**:

Podemos **redireccionar** la E/S usando:

- `>` cambia el fichero stdout
- `<` cambia el fichero stdin
- `<<, >>` igual pero si el fichero objeto ya existe no se borra y se añade al final
- `|` stdout del primero = stdin del segundo.
- Si se añade un 2 antes de estos símbolos se redireccionará al error estándar

5.2 Script

Un **script** o programa shell es un **fichero de texto** conteniendo una secuencia de **comandos** de shell externos e internos que se ejecutan **línea a línea**. `#!/bin/bash` (shebang) indica el **intérprete de**

comandos a usar por el script, es recomendable usarlo al principio, pero no es obligatorio. En `/etc/shells` hay una lista con la ruta completa de los shells disponibles

Se pueden ejecutar (siempre que tengan permisos con):

- `./script.sh`
- `bash script.sh`

5.3.1 Parámetros y Variables

Los **argumentos** de los scripts se pasan por la línea de comandos `./script.sh [parametros]`

- `$0` nombre del script
- `$1-$9` parámetros del 1 al 9
- `${10},...` parámetros por encima del 9
- `$#` número de parámetros
- `$*` y `$@` todos los parámetros

Se pueden leer parámetros de un fichero o el teclado con `read`. Se puede usar variables de entorno directamente dentro del script.

- `$?` código de salida del comando anterior
- `$$` PID del script actual
- `nombre=$(comando)` asigna la salida de un comando a una variable

5.3.2 Control de Flujo

```
for variable in lista
do
    bloque de comandos usando $variable
done

for ((a=0; a<5; a++))
do
    sufijo="0$a"
    touch servidor_${sufijo}.data
done

while comando
do
    bloque de comandos
done

until comando
```



```
do
    bloque de comandos
done
```

- `break` : para salir de un lazo
- `break n` : para salir de n lazos
- `continue` : para saltar a la siguiente iteración de un lazo

```
if comando1
then
    ejecuta otros comandos
elif comando2
then
    ejecuta otros comandos
else
    ejecuta otros comandos
fi

case valro in
    patron1)
        bloque de comandos
        ;;
    patron2)
        bloque de comandos
        ;;
    *)
        bloque de comandos por defecto
        ;;
esac
```

- `test expresion o [expresion]` devuelve 0 como código de salida si la `expresion` es verdadera y 1 si es falsa
 - `-d` comprueba si el parámetro es un directorio
 - `-e` se ejecuta con todo (significa existe)
 - `-f` comprueba si el parámetro es un fichero regular
 - `=, !=` comparación entre strings
 - `-eq, -gt` comparación entre enteros
 - `!, -a, -o` : not, and y or
 - `\(... \)` agrupación de expresiones

5.3.3 Funciones

Podemos organizar el código de un script definiendo funciones:

- Hay que definirla antes e usarla
- Los parámetros se almacenan por orden en las variables \$n
- el código de salida de la función se especifica con: `return codigo`

```
nombreDeLaFuncion(){  
    comandos  
}
```